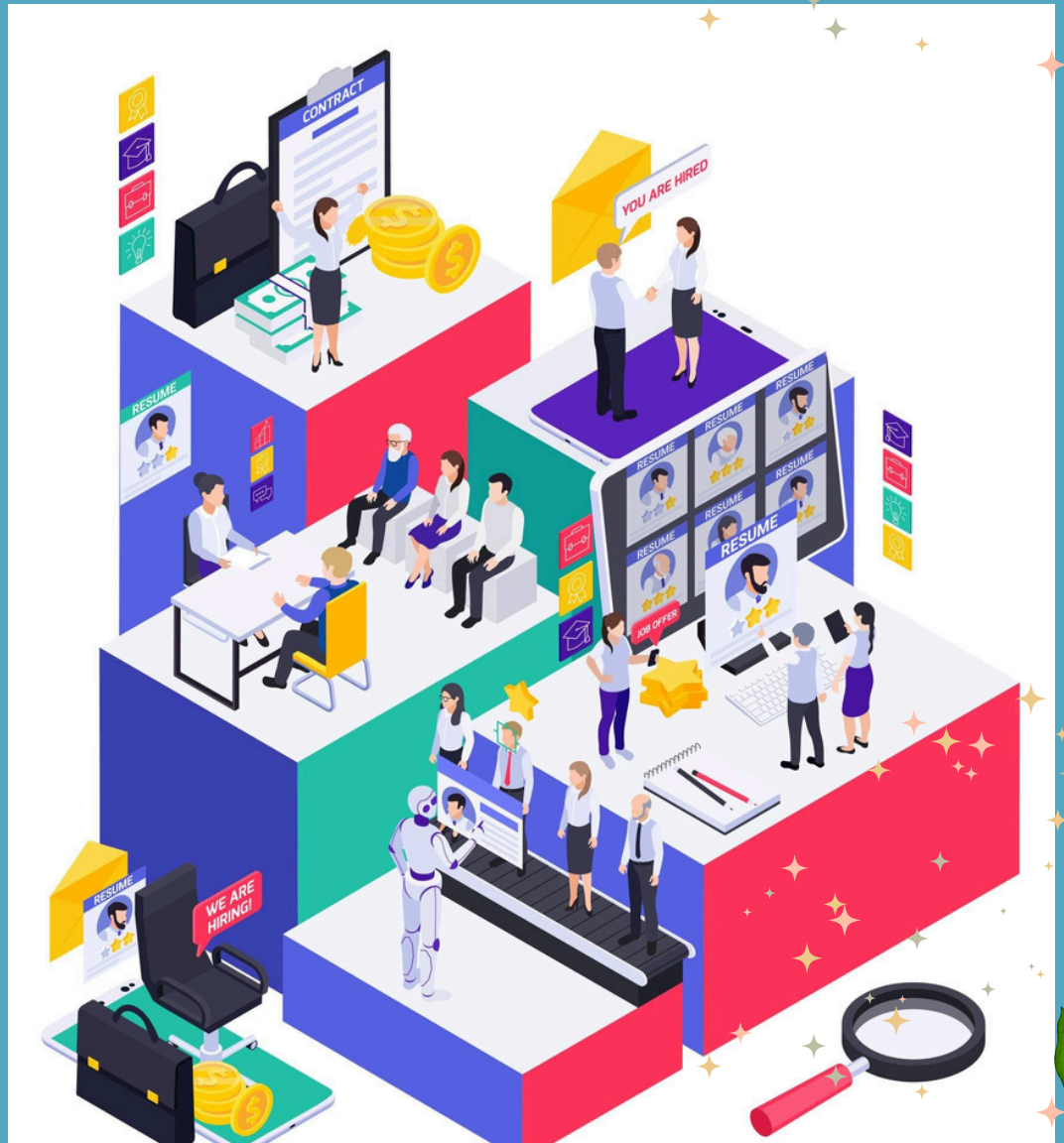
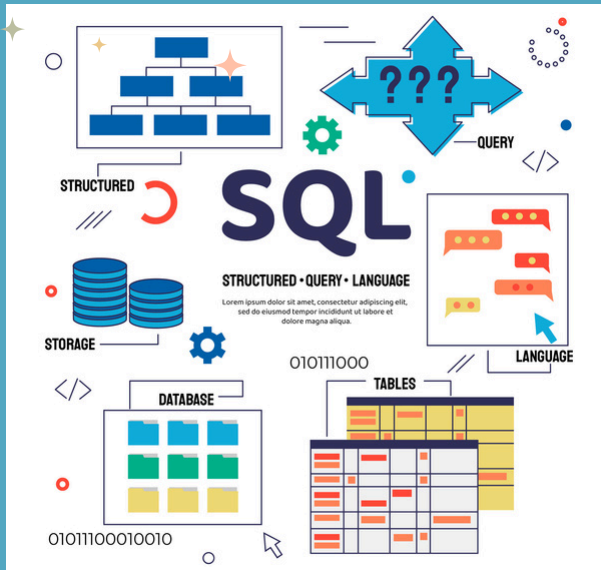


Employee Management System





Introduction

The Employee Management System is an SQL-based project that streamlines managing employee and department data, tracks performance reviews, and generates insights through queries. It uses key SQL concepts like database creation, table relationships, joins, and window functions to analyze employee performance, salary, and department metrics. This system ensures efficient data management and provides insights for strategic HR decision-making.



Schema

```
CREATE TABLE Employees (
```

```
    EmployeeID INT PRIMARY KEY,
```

```
    FirstName VARCHAR(50),
```

```
    LastName VARCHAR(50),
```

```
    DateOfBirth DATE,
```

```
    HireDate DATE,
```

```
    JobTitle VARCHAR(100),
```

```
    Department VARCHAR(100),
```

```
    Salary DECIMAL(10, 2),
```

```
    Gender VARCHAR(10)
```

```
);
```

```
-- Insert some sample data
```

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, DateOfBirth, HireDate, JobTitle, Department, Salary, Gender)  
VALUES
```

```
(1, 'John', 'Doe', '1980-01-15', '2010-05-01', 'Software Engineer', 'IT', 80000.00, 'Male'),
```

```
(2, 'Jane', 'Smith', '1985-07-22', '2012-03-15', 'Marketing Manager', 'Marketing', 85000.00, 'Male'),
```

```
(3, 'Michael', 'Johnson', '1990-11-30', '2018-08-20', 'Sales Representative', 'Sales', 60000.00, 'Male'),
```

```
(4, 'Emily', 'Davis', '1975-02-25', '2005-09-10', 'HR Specialist', 'HR', 70000.00, 'Female'),
```

```
(5, 'Sarah', 'Brown', '1992-05-05', '2020-12-01', 'Financial Analyst', 'Finance', 75000.00, 'Female');
```



Adhoc Request

```
select * from employees;
```

#1. Find employees who earn more than the average salary in their department#

#2. Find the second highest salary in the company#

#3. Select employees who were hired in the same year as the employee with EmployeeID = 1

#4. Find employees who earn more than the average salary#

#5. List employees with a salary in the top 10% of all salaries#

#6. Identify employees hired in the same year as another employee#

#7. Display the employee with the highest salary in each department#

#8. Stored Procedure to Calculate Bonus for Employees Based on Salary#

#9. Trigger to Automatically Update an Audit Table When Salary is Updated#

#10. Indexed Query to Optimize Searching by Department and Salary#

#11. Stored Procedure to Find the Top N Salaries in Each Department#



1. Find employees who earn more than the average salary in their department

```
1 SELECT *
2 FROM Employees e
3 WHERE Salary > (
4     SELECT AVG(Salary)
5     FROM Employees
6     WHERE Department = e.Department
7 );
8
```

Result Grid		Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:	
EmployeeID	FirstName	LastName	DateOfBirth	HireDate	JobTitle	Department	Salary	Gender	
2	Jane	Smith	1985-07-22	2012-03-15	Marketing Manager	Marketing	90000.00	Female	
6	David	Wilson	1988-03-12	2016-06-25	Product Manager	Product	90000.00	Male	
7	Olivia	Taylor	1984-10-09	2010-05-01	Graphic Designer	Design	95000.00	Male	
9	Sophia	Thomas	1982-06-21	2013-09-01	Legal Advisor	Legal	95000.00	Male	
13	Mia	Clark	1986-09-03	2011-02-14	Administrative Assistant	Administration	58000.00	Male	
16	Liam	Miller	1989-04-15	2015-07-20	Data Scientist	Data Analytics	92000.00	Male	



2. Find the second highest salary in the company

```
1 SELECT MAX(Salary) AS SecondHighestSalary
2 FROM Employees
3 WHERE Salary < (SELECT MAX(Salary) FROM Employees);
4
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

SecondHighestSalary

120000.00



3. Select employees who were hired in the same year as the employee with employeeID=1

```
1 SELECT *
2 FROM employees
3 WHERE YEAR(hiredate) = (
4     SELECT YEAR(hiredate)
5     FROM employees
6     WHERE employeeid = 1
7 );
```

Full Grid |  |  Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

EmployeeID	FirstName	LastName	DateOfBirth	HireDate	JobTitle	Department	Salary	Gender
1	John	Doe	1980-01-15	2010-05-01	Software Engineer	IT	80000.00	Male
3	Michael	Johnson	1990-11-30	2010-05-01	Senior Sales Representative	Sales	60000.00	Male
7	Olivia	Taylor	1984-10-09	2010-05-01	Graphic Designer	Design	95000.00	Male
11	Emma	Lee	1987-07-18	2010-05-01	Business Analyst	Business Intelligence	95000.00	Female
19	Mason	Clark	1985-11-25	2010-05-19	Project Manager	Operations	95000.00	Male
32	Zoe	Campbell	1985-06-23	2010-01-28	Senior Software Engineer	IT	95000.00	Male



4. Find the employees who earn more than the average salary

```
1 SELECT *
2 FROM Employees
3 WHERE Salary > (SELECT AVG(Salary) FROM Employees);
4
```

Result Grid									
Filter Rows:									
Edit:   									
Export/Import:  									
Wrap Cell Content: 									
EmployeeID	FirstName	LastName	DateOfBirth	HireDate	JobTitle	Department	Salary	Gender	
1	John	Doe	1980-01-15	2010-05-01	Software Engineer	IT	80000.00	Male	
2	Jane	Smith	1985-07-22	2012-03-15	Marketing Manager	Marketing	90000.00	Female	
6	David	Wilson	1988-03-12	2016-06-25	Product Manager	Product	90000.00	Male	
7	Olivia	Taylor	1984-10-09	2010-05-01	Graphic Designer	Design	95000.00	Male	
9	Sophia	Thomas	1982-06-21	2013-09-01	Legal Advisor	Legal	95000.00	Male	
11	Emma	Lee	1987-07-18	2010-05-01	Business Analyst	Business Intelligence	95000.00	Female	



5. List the employees with a salary in the top 10% of all salaries

```
1 WITH salary_percentile AS (  
2     SELECT  
3         salary,  
4         NTILE(10) OVER (ORDER BY salary DESC) AS salary_decile  
5     FROM employees  
6 )  
7 SELECT e.*  
8 FROM employees e  
9 JOIN salary_percentile sp ON e.salary = sp.salary  
10 WHERE sp.salary_decile = 1;  
11
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

	EmployeeID	FirstName	LastName	DateOfBirth	HireDate	JobTitle	Department	Salary	Gender
	7	Olivia	Taylor	1984-10-09	2010-05-01	Graphic Designer	Design	95000.00	Male
	9	Sophia	Thomas	1982-06-21	2013-09-01	Legal Advisor	Legal	95000.00	Male
	11	Emma	Lee	1987-07-18	2010-05-01	Business Analyst	Business Intelligence	95000.00	Female
	19	Mason	Clark	1985-11-25	2010-05-19	Project Manager	Operations	95000.00	Male
	32	Zoe	Campbell	1985-06-23	2010-01-28	Senior Software Engineer	IT	95000.00	Male
	35	James	Gray	1988-10-21	2013-06-12	IT Director	IT	110000.00	Male



6. Identify employees hired in the same year as another employee

```
1 • SELECT DISTINCT e1.*
2 FROM employees e1
3 JOIN employees e2
4     ON YEAR(e1.hiredate) = YEAR(e2.hiredate)
5     AND e1.employeeid <> e2.employeeid
6 ORDER BY YEAR(e1.hiredate);
7 |
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content: 

EmployeeID	FirstName	LastName	DateOfBirth	HireDate	JobTitle	Department	Salary	Gender
77	Alice	Johnson	1980-04-12	2010-03-15	Manager	Management	95000.00	Female
54	Logan	Griffin	1985-11-02	2010-08-28	Chief Financial Officer	Finance	135000.00	Female
39	Jackson	Cook	1983-01-20	2010-06-30	Sales Manager	Sales	88000.00	Male
32	Zoe	Campbell	1985-06-23	2010-01-28	Senior Software Engineer	IT	95000.00	Male
19	Mason	Clark	1985-11-25	2010-05-19	Project Manager	Operations	95000.00	Male
11	Emma	Lee	1987-07-18	2010-05-01	Business Analyst	Business Intelligence	95000.00	Female

7. Display the employee with the highest salary in each department

```
1 SELECT EmployeeID, FirstName, LastName, Department, Salary
2 FROM Employees e1
3 WHERE Salary = (SELECT MAX(Salary)
4                 FROM Employees e2
5                 WHERE e2.Department = e1.Department);
6
```

Result Grid | Filter Rows: | Edit: | Export/Import:

EmployeeID	FirstName	LastName	Department	Salary
2	Jane	Smith	Marketing	90000.00
6	David	Wilson	Product	90000.00
7	Olivia	Taylor	Design	95000.00
11	Emma	Lee	Business Intelligence	95000.00
13	Mia	Clark	Administration	58000.00
16	Liam	Miller	Data Analytics	92000.00



8. Stored procedure to calculate bonus for Employees based on Salary.

Name: calculate_bonus

The name
statement

DDL:



```
1 CREATE DEFINER='root'@'localhost' PROCEDURE `calculate_bonus`()  
2 BEGIN  
3     SELECT firstname, lastname, salary,  
4         CASE  
5             WHEN salary > 70000 THEN salary * 0.10  
6             ELSE salary * 0.05  
7         END AS bonus  
8     FROM employees;  
9 END
```



9. Trigger to Automatically update an audit table when salary is updated

```
CREATE TABLE salary_audit (  
    audit_id INT AUTO_INCREMENT PRIMARY KEY,  
    employeeid INT,  
    old_salary DECIMAL(10, 2),  
    new_salary DECIMAL(10, 2),  
    update_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

10. Indexed query to optimized searching by Department and Salary

```
1 • CREATE INDEX idx_department_salary ON employees(department, salary);
2
3
4 • SELECT firstname, lastname, department, salary
5 FROM employees
6 WHERE department = 'HR'
7 ORDER BY salary DESC;
8
```

Result Grid |  |  Filter Rows: | Export:  | Wrap Cell Content: 

firstname	lastname	department	salary
Victoria	Bailey	HR	90000.00
Olivia	Nelson	HR	80000.00
Sophia	Reed	HR	68000.00
Sophia	Reed	HR	68000.00
Sophia	Reed	HR	68000.00
Grace	Roberts	HR	67000.00

10. Indexed query to optimized searching by Department and Salary

Name: top_n_salaries_by_department

The name of the routine is parsed automatically from statement. The DDL is parsed automatically while you

DDL:



```
1 • CREATE DEFINER=`root`@`localhost` PROCEDURE `top_n_salaries_by_department`(IN top_n INT)
2 BEGIN
3     SELECT firstname, lastname, department, salary
4     FROM (
5         SELECT firstname, lastname, department, salary,
6                ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
7         FROM employees
8     ) ranked
9     WHERE rnk <= top_n;
10 END
```

Conclusion

The Employee Management System demonstrates how SQL can be leveraged to efficiently manage and analyze employee and department data. By utilizing core SQL concepts such as database creation, table relationships, and advanced queries like window functions and joins, this system offers valuable insights into employee performance, salary distributions, and department metrics. This project showcases the powerful capabilities of SQL in organizing and extracting data, aiding in data-driven decision-making for HR and management.



For your time



Created by: Dhanesh Kumar Goan

